Mightling: A Vendor-Free Coding Agent on a Single Unified-Memory Workstation, Built on the Codex CLI

dgxcoder
dgxcoder@dreamference.ai

October 2026

Abstract

Terminal coding agents such as the Codex CLI are open source, but they are built around a hosted model, a vendor account and a cloud of companion services. We describe *Mightling*, a coding agent whose model runs on one NVIDIA GB10 workstation (128 GB of CPU–GPU unified memory): a 27B-parameter model (Qwen3.8-27B, NVFP4) served locally through SGLang with a block-diffusion drafter. Source code, prompts and inference never leave the machine, and no vendor account is involved. Mightling is a fork of the Codex CLI (release `rust-v0.158.0`, 4,894 Rust source files, 1.93 million lines). Its central engineering choice is to keep the fork almost identical to upstream. The upstream tree is never edited. Each build exports it, applies a series of 23 patches totalling 43.0 KB (43 files, $+170/-53$ lines), and links in a separate crate that does the actual work of localisation: model discovery, configuration, prompt rebranding, help-text rewriting, a signed release-based updater and refusal of cloud-bound commands. The localisation itself takes about 9 KB of the series; every feature beyond it costs one hook in Codex and puts its logic in a crate of ours. We describe the inventory of the cloud dependencies that are switched off, the trace-based audit that keeps them off, an enforced network switch, the integration points between a vendor harness and a local model that are not visible from the documentation, and the host-safety layer that unified memory requires. On a 100-task sample of SWE-bench Verified, run inside each task's container on this machine, the agent resolves 68; calibrated against the 175 published leaderboard submissions on the same tasks, that corresponds to about 69% on all of Verified (95% band 64–73%). A two-session "study, then fix" mode resolves 69 of the same 100 at 2.2 times the time, and four prompt and harness changes measured as paired A/B nights on fresh tasks show no measurable difference; three runs of one configuration on the same 44 tasks resolve 29, 29 and 31, and some run resolves 36. The fork also carries, each from the same launcher crate or beside it: a terse-answer mode that halves answers to questions and leaves coding work unchanged, a compaction limit tied to the server's KV pool with a rule-built ledger of what summaries lose, a two-layer code index (a tree-sitter knowledge graph plus compiler-exact SCIP data), an overnight task queue that works in git worktrees, and a client/node split that pairs workstations over SSH. We are explicit about what is estimated rather than measured.

1 Introduction

Agentic coding tools now read repositories, run shell commands and edit files autonomously [21–23]. The most capable are open-source clients of proprietary services: the client is inspectable, but a session sends source code, prompts and tool output to a hosted model, which regulated industries, security research and anyone whose code is the asset cannot accept. Two developments make a local alternative plausible: open-weight models quantised to four bits [6] with speculative decoding [4, 13] generate at interactive speed on one desk-side machine, and the leading agent clients are

open source, so a local deployment can reuse their tool-calling loop, sandboxing, approval flows and terminal UI. Reusing a fast-moving upstream has a cost: a fork that edits freely soon cannot take upstream releases, and one that edits nothing cannot remove the parts that assume a cloud. This paper describes how Mightling, a local AI stack for the NVIDIA GB10 made by Dreamference, resolves that tension in its terminal coding agent.

What "local" means here. The model, the agent and all session state run on one workstation; source code, prompts, tool output and model traffic never leave it, and a traced session shows no connection to any vendor (section 5.1). Mightling is not air-gapped: web search goes through a SearXNG [16] instance on the machine, mail is read over IMAP, `ling update` contacts GitHub, and the phone messengers reach their networks once turned on. Each can be switched off, and `/airgapped on` gives every command the agent runs an empty network namespace.

Contributions. A **patch-minimal fork method** for a large Rust agent (section 4): an unmodified pinned submodule, a build-time export, one- and two-line hooks, and an external crate linked into the binary, with the four techniques that keep the series at 43.0 KB, of which the localisation is about 9 KB. An **inventory of the cloud surface** of a coding-agent client and its treatment (section 5), a trace-based audit that re-checks it after every build, and a network switch enforced in the command sandbox. **Integration points** between a vendor harness and a local model (section 6): model discovery, a compaction limit the server's memory sets, and tool types the server silently drops. A **host-safety layer for unified memory** (section 7). A **measured cost model** for a local agent's output and the terse-answer mode built on it (section 6). A **task-level evaluation on SWE-bench Verified** on an arm64 desk machine (section 9) with a calibrated overall estimate, a classification of every failure, and paired A/B nights that measure what one night can detect. The first results of an inference engine for this one model, exact by rule, on a second workstation (section 10), the rest of the stack in brief (section 11) and the design principles behind it all (section 12).

2 Background

The Codex CLI. The Codex CLI [7] is an open-source Rust workspace of about 160 crates: a terminal UI, a non-interactive `exec` mode, session resume and fork, a platform sandbox (Bubblewrap on Linux) and an approval model for tool calls. In *Code Mode* the model is given a single `exec` tool and writes short JavaScript programs, run by a separate V8 host process (`codex-code-mode-host`), within which every other tool, including Model Context Protocol (MCP) [2] tools, is a function. Codex talks to open-source providers (`--oss`) through a chat-completions endpoint, but login, cloud tasks, connectors, usage limits and feedback upload assume an account with the vendor's hosted service.

The machine. The NVIDIA GB10 pairs an Arm CPU with a Blackwell GPU (compute capability 12.1) over 128 GB of shared LPDDR5X. No memory is reserved for the GPU: model weights, KV cache, the desktop and every other process draw from one pool, which shapes section 7.

The model. Qwen3.8-27B in an NVFP4 checkpoint, served by SGLang with a calibrated NVFP4 DFlash2 block-diffusion drafter [4, 25] proposing twelve tokens per step (7% faster than sixteen on replayed sessions; eight, ten and fourteen gain nothing), a 262k-token context, about 38 GB free while serving. Single-stream it decodes 32.0, 68.6 and 84.1 tokens/s on prose, code and JSON; replayed agent sessions decode about 48 tokens/s at a step of about 103 ms with 5.2 tokens accepted

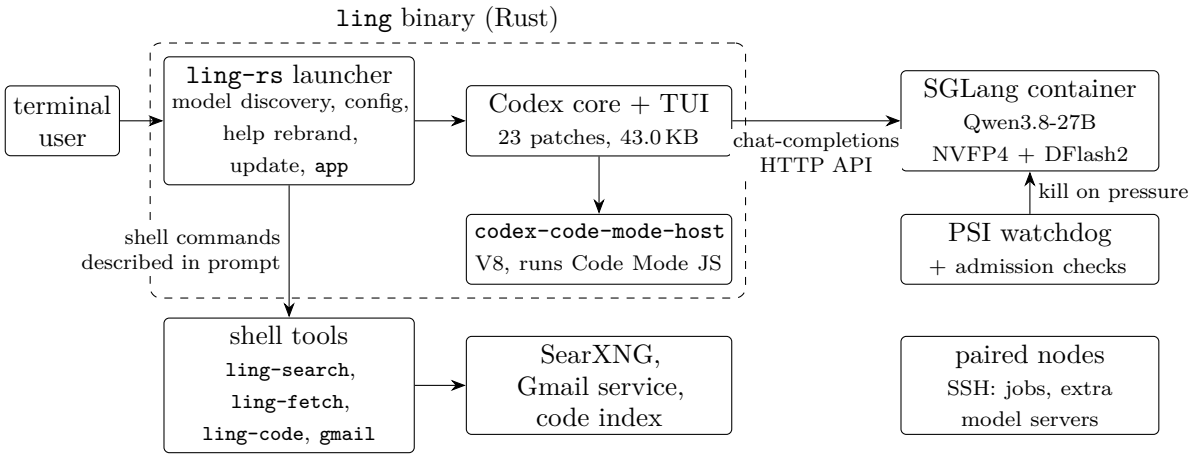


Figure 1: Mightling at run time. The only Codex code paths that differ from upstream are the hooks applied by the patch series; everything Mightling-specific lives in the linked `ling-rs` crate or in separate local services.

per step, and 33.9 tokens/s each on two streams. The server manager also drives vLLM [12], so the method does not depend on one engine.

3 System overview

Figure 1 shows the running system. The user types `ling`, a single Rust binary. Before Codex parses its command line, the linked launcher crate finds the local model server and waits for it if it is still loading, asks it which model it serves and how long its context is, writes the model catalogue and configuration Codex needs, and prepends the options that point Codex at the local provider. From then on the process is Codex, and every upstream flag and subcommand behaves as documented, apart from the cloud surfaces of section 5. Three small Rust programs beside `ling` give it web search, page fetching and code navigation (`ling-search`, `ling-fetch`, `ling-code`); `ling-admin`, the Python management CLI, provides Gmail reading and manages the model-server container, a browser chat UI built on Onyx [14], the overnight queue, workstation pairing, SWE-bench runs and the build of `ling`. The other surfaces (a desktop app, phone messengers, a local file index) are summarised in section 11.

4 A patch-minimal fork

4.1 Build pipeline

Codex is vendored as a git submodule of our own GitHub fork, pinned to the stable release tag `rust-v0.158.0`, and *never modified*: the builder's tests assert that `git status` inside it is empty after a build. A build exports the pinned `codex-rs/` with `git archive`, copies the launcher crate in as `codex-rs/ling`, applies the series with `git apply --check` before `git apply` so a stale patch fails before anything is half-applied, builds `ling` and `codex-code-mode-host` with Cargo, installs them side by side, and records a build key (submodule commit, patch bytes, launcher sources) so an unchanged key skips the build. Moving to a new upstream release means bumping the submodule and resolving whichever hunks stop applying; because the hunks are few and small, conflicts are

local and obvious.

4.2 Four techniques keep the series small

Table 1 shows the series. *Rename at run time, not in the tree*: renaming the agent inside Codex's bundled model catalogue would cost about 395 KB of diff, since each 20 KB system-prompt template is one JSON line; the launcher instead reads the catalogue through the upstream crate's public function, replaces the identity sentence, and hands the result to Codex in the catalogue it writes anyway. *Rewrite presentation where it is assembled*: one hook routes argument parsing through a launcher function that walks the `clap` tree once and rewrites the product name in every `about`, `help` and `bin_name`. *Use the build system's own unification*: one `openssl-sys = {features = ["vendored"]}` line in *our* manifest applies to every crate that links OpenSSL, and a path dependency inside the workspace root makes the launcher a workspace member without editing the workspace manifest. *One line per behaviour*: what remains is a `false` from a visibility predicate, a `hide = true`, or a dispatch arm sent into the launcher.

A feature costs a hook, not a diff. Patches 0001 to 0016, the localisation, are about 9 KB of removals and redirections. From 0017 on the patches are features, each with its logic in a crate of ours and only its entry point in Codex: `/cavemode` and `/night` add a slash-command variant and its dispatch arm (566 and 769 lines in the launcher with their tests); `/airgapped` is one call in each place a command's permissions are resolved; flattening MCP tools and observation masking each route one step of request assembly through a crate of ours; `/apps` answers Codex's connector list from the launcher; `/node` suspends the TUI so a password typed there never enters the session. A test caps the series at 43,000 bytes, raised only explicitly in the commit that needs it. Two build details need no patch: Code Mode's V8 flavour is not published for `aarch64-unknown-linux-gnu`, so the builder fetches upstream's own prebuilt archive and verifies it against the submodule's digest manifest [8], and two release-profile overrides take the binary from 1.4 GB to 315 MB.

5 Cutting the cloud cord

Every subcommand and slash command of Codex 0.158.0 was audited for a dependency on vendor-hosted services; table 2 lists the result under four treatments. *Refuse*: the launcher rejects the command before Codex parses it. *Hide*: the command leaves help and menus but stays compiled for a future private-cloud implementation. *Replace*: the same command does something local. *Remove*: the command is unreachable. Hiding alone is not a security boundary, since a hidden clap subcommand still parses, so the commands that start a vendor sign-in or browse the vendor's servers are also refused, by a check that reads Codex's own clap definitions and so refuses `ling -c k=v cloud` as well.

5.1 Traffic an audit of commands does not find

Auditing commands finds the channels a user starts deliberately and none of those Codex opens by itself. Those are found by tracing real sessions under `strace -f -e trace=connect,execve` with the HTTP client's debug logging on; with the model on loopback, anything else in the trace is unintended. An upstream build with no vendor login opens four channels: OpenTelemetry metrics to Statsig at `ab.chatgpt.com`, the default in release builds, with an API key compiled in; a `git ls-remote` of the vendor's plugin repository; a request for featured plugins to `chatgpt.com`; and an announcement fetch from `raw.githubusercontent.com` on every interactive start. Two more

Patch	Files	+/−	Purpose
0001 brand-mightling-name	12	20/19	Product name in TUI header, status card, slash-command list, `exec` banner and reply label, fixed usage strings
0002 ling-launcher-hook	2	2/1	One dependency line; route argv through the launcher
0005 hide-vendor-login	2	4/0	Hide `login`/`logout` and `/logout`
0006 hide-cloud	1	2/1	Hide `cloud` (kept for a private cloud)
0007 hide-remote-control	1	2/0	Hide `remote-control`
0008 mightling-update	1	2/2	Send `update` to the launcher's release updater
0009 remove-feedback	2	3/1	Remove `/feedback` (uploads session logs)
0010 hide-voice	1	4/1	Hide `/voice` (hosted realtime API) and `/pets` (art fetched from the vendor's CDN)
0011 usage-token-stats	4	7/17	`/usage` shows local token statistics
0012 hide-auto-review	1	2/0	Hide `/approve` (defaults to a hosted review model)
0013 disable-usage-analytics	1	2/1	Construct the analytics client disabled
0014 mightling-home	1	2/0	Use `~/.mightling`, never upstream's login
0015 no-vendor-network	4	20/4	Close the channels found by tracing (§5.1); drop Codex's own tips
0016 doctor-offline	2	20/0	Switch off `doctor`'s sign-in, update and reachability checks
0017 cave-mode	5	10/0	`/cavemode`; register the terse-answer section the launcher supplies
0018 night-slash-command	2	7/0	`/night`: queue a task for the overnight run
0019 airgapped	8	17/2	`/airgapped`; enforce `on` in the Linux sandbox helper; Full Access refused at `on`
0020 flat-mcp-tools	3	4/1	Send MCP tools, and Code Mode's `exec` where the server would drop it, as plain functions (section 6)
0021 observation-masking	2	5/1	Optionally replace old tool outputs by a placeholder naming a saved copy
0022 ling-apps	2	6/1	`/apps` lists local Gmail, Drive and Calendar connectors, no vendor sign-in
0023 airgapped-app-server	2	2/0	The app server refuses Full Access at `on` for every client
0024 airgapped-windows	4	12/0	The same enforcement in the Windows sandbox
0025 node-slash-command	4	15/1	`/node`: provision, pair and check nodes from the TUI (section 11)
Total (23 patches, 42,958 bytes)	43	170/53	against 4,894 Rust files, 1.93M lines

Table 1: The patch series. Numbers 0003 and 0004 are unused. The total counts each upstream file once; the per-patch column counts it once per patch.

Surface	Why it does not fit a vendor-free agent	Treatment
`login`, `logout`, `/logout`	Vendor-account / API-key sign-in; no account exists	refuse + hide
`cloud`, `cloud-tasks`	Browses tasks on the vendor's servers	refuse + hide
`remote-control`	Relays sessions through the vendor's servers	hide
`update`	Installs upstream Codex over Mightling	replace (verified release download)
`/feedback`	Uploads session logs to the vendor	remove
`/voice`	Hosted realtime voice API	hide
`/usage`	Hosted-plan usage and limit resets	replace (local token statistics)
`/approve` (auto-review)	Defaults to a hosted review model	hide
`app`	Opens the vendor's closed-source desktop app	replace (opens Mightling's own window)
update check at start-up	Compares against upstream Codex releases	disabled in config
Found only by tracing a session (§5.1)		
usage analytics	Posted to `chatgpt.com` whenever a vendor login is present	disabled
shared `~/.codex`	Inherits an upstream vendor login and its token refresh	own `~/.mightling`
OTEL metrics	Exported to Statsig (`ab.chatgpt.com`) by default in release builds	disabled
curated plugin sync	`git ls-remote` of the vendor's plugin repository on GitHub at start-up	removed
featured plugins	Request for featured plugins to `chatgpt.com`	removed
announcement tip	TUI fetch from GitHub's content host on every start	removed

Table 2: Cloud-coupled surfaces and their treatment. On Linux, Codex's `app` subcommand is not compiled at all, so Mightling's `app` is new.

open whenever a login is present in the shared `~/.codex`: usage analytics and token refresh. Each is closed where it originates, so no setting or later login can reopen it; Mightling keeps its state in `~/.mightling`, never the login; and the launcher points Codex's `chatgpt_base_url` at a closed local port, so any vendor-backend call we did not find fails on the machine. What a vendor client sends is an empirical question, re-measured after every upstream update.

The audit as a command. That trace is `ling-admin audit egress`, run by every build after installing a new binary: one real session in a throwaway repository and home under `strace -f -e trace=connect,sendto,sendmsg,sendmmsg,execve`, passing only if every destination is an allowlisted loopback port, no DNS query was sent and no git command reached a network; a trace that itself failed is never a pass. glibc sends name lookups with `sendmmsg`, so tracing `connect` alone misses a query, and a connection to the stub resolver at `127.0.0.53:53` is loopback yet still a query that leaves the machine. A messenger bridge talks to its network outside the agent's process, so every report names each bridge that is on as a declared exception.

An enforced network switch. At `/airgapped on`, every command the agent runs gets an empty network namespace. The hook is in Codex's sandbox helper, not its core: core derives a command's network policy in about a dozen places, while every sandboxed command passes through the helper. The level resolves from the session, then an environment variable, then the stricter of the repository's and the user's configuration, because the agent can write the former. Full Access has no sandbox, so the launcher, the permission menus and the app server each refuse the combination.

Updates. `ling update` downloads the latest published release's binaries with a SHA-256 manifest, verifies both before replacing either, and swaps them in next to the running executable. Releases are *signed*: the checksum file carries an Ed25519 signature made in a protected CI environment, the public key is compiled into `ling`, and the updater and the installer refuse a release whose signature is missing or does not verify; the key is usable only from the main and release branches.

6 Running a vendor agent against a local model

Discover, do not configure. The server's `/v1/models` reports the served model and its `max_model_len`, and the launcher derives the catalogue entry from it rather than recording which model is served. Codex parses catalogue entries with strict `serde` structs, and `base_instructions` is the *entire* system prompt: with a placeholder there the model has no description of its tools and falls back to `curl`, so the launcher supplies Codex's own template with the identity sentence rebranded. Codex's `config.toml` accumulates tables Codex itself writes, so the launcher edits it as a syntax tree rather than appending keys that silently join the last table.

The compaction limit follows the server, not the model. The model reports a 262,144-token context, but the server's KV-cache pool, sized at launch from free memory and shared by every stream, holds about 157,000 tokens, so a compaction limit derived from the window would exceed what the server can hold; and a compaction is expensive, since the model re-runs what the summary dropped (at an extreme limit of 6,000, one small task compacted seven times and ran thirteen commands where three were needed). Two mechanisms follow, neither a patch. The launcher reads the pool from the server's metrics at every start and passes 60% of it as the compaction limit unless the user set one (94,144 tokens here). And a ledger replaces what summaries lose: Codex's summary keeps the prose but drops every tool call and output, so a hook run after each compaction has the

launcher rebuild by rule, with no model, in about 15 ms, the files the session touched, the commands that failed since the last compaction and the last test result, and hand them back as context. On one long task at a 32k limit, the session with the ledger compacted 11 times instead of 24 and ran 114 commands instead of 245; after a compaction only 1% of commands repeat an earlier one exactly. Benchmark sessions, at a 49,152-token limit, compact far more: across seven runs of 320 tasks, tasks that compacted at most once resolved 75–78%, twice 57%, and three times or more 38% (50% against 40% on the 18 tasks that ran both ways), and compaction took 17% of all agent time at a median of 98 s. Compacting at the end of a turn instead of mid-turn is built as a setting and off until a benchmark of interactive sessions decides it.

Tools as shell commands, and tool types at the boundary. The agent's primary tools are shell commands described in the system prompt (`ling-search`, `ling-fetch`, `ling-code`, `gmail`), which a local model calls reliably and which need no tool type the server has to understand. Codex sends two kinds of tool a local server drops: MCP tools go out in a `namespace` tool type of Codex's own, and Code Mode's `exec` and the free-form `apply_patch` go out as *custom* tools whose argument is raw text under a grammar. Both SGLang and vLLM drop tools of a type they do not know, so none of them reaches the model as sent. Patch 0020 sends each MCP tool as a plain function; on a server other than our own engine, the launcher's tool crate offers `exec` as a function with one string parameter, the grammar in its description, turns the model's call back into the custom-tool call Codex expects before Codex records it, and replays earlier calls in the form the model saw. With that, the model calls `exec` in every one of 44 benchmark tasks, 2,865 times; the `apply_patch`-as-function arm never started, because the pinned Codex's catalogue parser accepts only the free-form variant (section 9). The request that reaches the model, not the one the harness builds, is the thing to inspect. Codex's only pre-token timer, a 300 s stream-idle timeout, covers the server's queue as well, which several queued streams exceed, so the launcher writes 900 s unless the user set one.

Prose is the expensive output. Locally the currency is time, and its exchange rate is lopsided: this model decodes prose at 25–32 tokens/s and reads a prompt at about 1,700, so one token of prose costs the time of about 50–70 prompt tokens, and 81% of recorded input comes from the prefix cache. Standing instructions are nearly free; every sentence of answer is not. Across 52 recorded sessions, 75.5% of output tokens are tool-call arguments, 23.1% final answers and 1.3% commentary. Mightling therefore answers tersely by default. A *cave mode* level (`off`, `lite`, `full`, `ultra`; `/cavemode` switches it) is a section of model-visible session state the launcher contributes through Codex's extension registry, the level's text once and a one-line reminder each later turn; the system prompt is untouched, so `off` is exactly upstream. In 532 single-turn runs and 11 sessions of eight or nine turns, the default `ultra` (at most three sentences of prose outside code unless asked) cut answers to questions to 0.42–0.57 of their length at `off` and wall time to 0.41–0.74, passed 35 of 36 correctness checks, the same as `off`, and moved nothing on coding tasks, as a paired study of the public "caveman" skill found on a hosted agent [17]. Content rules (answer first, one option, no unrequested code, a numeric sentence cap) work where adjectives such as "short" barely do, and a level fades unless repeated: with the per-turn reminder, turns five to eight sum to 0.49 of `off`. Only what the model writes is compressed, since compressing what it reads makes models answer longer and worse [1]. Email, a prompt-injection channel for an agent holding a shell, is read-only, framed between *BEGIN EMAIL (untrusted)* and *END EMAIL* markers, and declared never to be instructions; a mitigation, not a defence.

7 Host safety on unified memory

With no dedicated GPU memory, a model load competes with the desktop for every page. A load that outruns reclaim takes the workstation down hard enough to need the power button, without invoking the kernel OOM killer: driver-pinned pages are unreclaimable and charged to no process the killer would select, so the kernel spins in direct reclaim. The server manager enforces two layers. *Admission checks before a load*, reported together because each fix needs its own `sudo`: an OOM handler (systemd-oomd or earlyoom) acting on pressure-stall information [19]; at least 64 GB of swap; `vm.min_free_kbytes` at 1 GiB or more, since the stock 44 MB is exhausted between reclaim passes by a loader reading gigabytes per second; `vm.watermark_scale_factor` at 200 or more; and sysstat, whose history is the only pressure record that survives a hard reset. The container is the preferred OOM victim (`oom_score_adj` = 800), and `ling-admin host setup` applies what the checks report, with an AppArmor profile that lets Bubblewrap create the sandbox's user namespace. *A watchdog during a load* samples `/proc/pressure/memory` once per second and kills the container when the "full" stall share over 10 s stays at or above 60% for 5 s (systemd-oomd's default) or the 60 s average reaches 25%, which catches pressure that oscillates around the first threshold. The kill tries three paths in the order the host can still afford: a direct `SIGKILL` to the cgroup, one request over Docker's Unix socket with no fork, and the Docker CLI last, because forking a Go binary is the work least likely to be scheduled during a reclaim livelock. The margins decide the context a model can run at, and anything resident beside the server starts before it with a fixed cgroup cap, because the server's pre-flight reads current free memory. Every heavy background job runs under `systemd-run -p MemoryMax=... -p MemorySwapMax=0`, so exhaustion kills the job, not the host; index runs share one slice and one memory ledger, each overnight task has a scope of its own, and while a night run holds its lock a model load, a build and a re-index all refuse to start.

8 Evaluation

Table 3 summarises what the system does today; task-level accuracy is the subject of section 9. A live suite drives every slash command of the pinned Codex on a pseudo-terminal against the running local model and asserts that the expected screen or reply appears and that removed commands are rejected. Decode speed for code is about twice that for prose and JSON is faster still, because the drafter's acceptance rises from 3.4 tokens per step on prose to 4.4 on code and 5.9 on JSON; this suits an agent whose output is three quarters tool-call arguments.

9 Task-level accuracy on SWE-bench Verified

Running it on a desk machine. `ling-admin swe-bench` runs `ling` inside each instance's container on the GB10 and grades the patches with the official harness [11]. The official images are `linux/amd64` only; the arm64 images are a community build covering 400 of the 500 instances [10], and an instance is used only once its gold patch resolves and a no-op patch does not, both checked here. The installed `ling` is relinked against the host's loader for the images' older glibc. Codex's sandbox cannot start inside a container, so the container is the sandbox, on an internal network that reaches the model server and nothing else. The server sits behind a streaming proxy, the *model gate*: while a run holds it, a request from outside the run's network gets a 503 naming the run and the time left. Each instance gets a 45-minute limit, 8 GB of memory and four CPUs; two run at once; one session, one patch, a single attempt in the leaderboard's sense.

Metric	Value
Upstream (Codex `rust-v0.158.0`)	4,894 Rust files, 1.93M lines
Patch series	42,958 B, 23 patches (localisation ≈9 KB)
Upstream files touched / lines changed	43 files / +170, −53
Launcher crates (`ling-rs` and its sub-crates)	35,900 lines, 553 tests
Code-index router (`ling-code`)	10,444 lines, 131 tests
Agent binary, unstripped → stripped	1.4 GB → 315 MB (+ 93 MB Code Mode host)
Incremental rebuild after a launcher edit	≈7 min (thin LTO relink)
Management and services (Python)	155 modules, 45.5k lines; 1,269 tests
Live slash-command suite, Qwen3.8-27B†	77 tests: 75 pass, 2 skip; 651.3 s
Decode throughput, single stream, 262k context	prose 32.0 / code 68.6 / JSON 84.1 tok/s
Decode, replayed agent sessions, 1 / 2 streams	48.3 / 33.9 tok/s each; 5.2 of 12 accepted
SWE-bench Verified, 100-task sample (section 9)	68 resolved; ≈69% overall (64–73%)
Code-index query, warm, this repository (20 runs)	p50 0.09 s, p95 0.11–0.12 s
Overnight task, live (one task, section 11)	7 s; test passed; checkout untouched

Table 3: Measurements on one GB10 (128 GB). Sizes are binary megabytes. Single-stream throughput at twelve draft tokens; the replay rows were measured on the second GB10 with production's image and flags. †Run on an earlier 16-patch build of the series; not yet re-run on the current one.

From a local subset to an overall score. A sample is not average: across the 175 submissions that publish per-instance results [18], systems resolve on average 5.9 points fewer of our 24-task sample than of all 500. Two estimators fitted on those submissions map a local count to an overall score: a regression of overall on subset score, and a one-parameter item-response (Rasch) model fitted on the full 175×500 matrix. Back-tested on the published submissions' own 24-task results, the Rasch estimate has an RMSE of 5.3 points (the regression 5.1), and the 95% band is ±1.96 times that error; on 100 tasks the error falls to 2.4 points, a band of about ±5.

Results. On the 100-task sample the default configuration resolves 68, about 69% on all of Verified (band 64–73%). On the 24-task sample six rounds resolve 17, 16, 17, 15, 16 and 15; the median, 16, corresponds to about 71% (band 60–81%) and is what several published systems on much larger hosted or open models score on exactly those tasks, with overall scores between 66.6% and 77.6%; the best published result on them from a 20B–40B model is 15 with several attempts, 14 as a single attempt, all 2025 models. A *refine* mode runs two sessions, one that studies the task without changing it and writes a structured description, and a fresh one that fixes it; paired with the default on the 100 tasks it resolves 69, each mode alone resolving about ten tasks the other misses ($p = 1.0$), at 2.2 times the time per task (median 16 min 22 s against 4 min 7 s on the 24-task sample). Refine is opt-in.

Why the 32 failed. Read against each task's difficulty, the share of the 78 published submissions scoring 60% or more that solve it, nineteen of the 32 failures are tasks fewer than 40% of those systems solve. By category: a second code path or file missed (8), the wrong file (5), the right place but a wrong detail (5), behaviour the issue did not ask for (5), a fix that broke other tests (4), the benchmark's test patch blocked by the agent's own test edits (2), stopped mid-work (2) and out of time (1). None came from an empty patch, a tool or environment error, or a malformed tool call. In four of the seven failed tasks that at least 75% of those systems solve, the source change was in the right place and a test decided the outcome.

What one night can measure. Changes to the task prompt, the harness and the tools of section 6 are measured as paired A/B nights on tasks outside the 100: two arms on the same 44 or 50 freshly validated tasks, one session per task.

Night	Candidate arm	Tasks	Default	Candidate (p)
1	three sentences of test discipline	50	34	33 (1.0)
2	the same, second sample	44	29	31 (0.73)
3	the issue as a contract to check	44	29	26 (0.51)
4	restart after a second compaction	44	31	28 (0.45)
5	`apply_patch` offered as a function	44	29	void (see below)
6	Code Mode off: `exec` left to the server to drop	44	31	30 (1.0)

None of the five that ran differs measurably from the default, which stays. The default arm's own runs on the same 44 tasks say why: they resolve 29, 29 and 31, only 23 tasks are resolved every time, and 36 by at least one of the three. A quarter of a run's resolved set is the luck of that run, so a single night can confirm only a large effect, and the arms are judged first by the behaviour they target. The same runs bound what choosing among attempts could gain: a perfect chooser over the three would resolve 36 of 44, about 14 points above a single run, at three times the agent time.

Night 5 is void. Its candidate arm failed at start in all 44 tasks: the launcher wrote `function` as the `apply_patch` tool type, the pinned Codex's parser accepts only `freeform`, every session ended within seconds with no tokens, and the round still exited 0, showing 0 of 44 with 44 agent errors only in its final report. Every night now begins with one real task per arm, and the unit does not start unless each session ran to its end (a wrong patch passes; a dead session does not). The default arm, whose tools were night 6's candidate's, resolved 29.

Night 6: Code Mode. With `exec` offered as a function the default arm resolved 31 of 44 at a median of 7 min 7 s per task with six timeouts; with `exec` left as the custom tool the server drops, the candidate resolved 30 at 5 min 38 s with one timeout (+2.3 points for Code Mode; McNemar $p = 1.0$). Code Mode stays on; the time is the finding. A request costs the same in both arms (5.03 against 5.01 s), and the Code Mode arm makes 11% more of them (6,353 against 5,738): 221 of its 2,864 `exec` cells (7.7%) failed before running anything (Python heredocs inside a JavaScript string, a nested tool called without its `tools.` prefix), each a full round trip, 12% of cells right after a compaction against 6.9% elsewhere; 2,646 cells (92%) wrapped exactly one shell command at 2.80 s per call against 2.43 s direct; and `exec`'s description adds about 1,000 tokens to the fixed prefix, costing five more compactions (60 against 55). Of the six timeouts, two trace to failed cells after a compaction, one is unclear, three would have happened anyway. The harness's report sums tokens over completed turns, which a timed-out session never has, so its 111M against 160M input tokens is wrong; from the rollouts the arms used 194.7M with Code Mode against 174.0M without (+12%). No task used what Code Mode adds (11 cells composed two tools). The remedies, guidance in `exec`'s description on when a direct call is right, restating the convention after a compaction, and naming the fault when a cell fails, are each an arm to measure.

These are estimates of what this agent, model and machine would score on all 500 tasks under these conditions, not leaderboard results: they do not account for training-data contamination, the quantised model, the community images and our validation filter, or run-to-run variation of several points.

10 An engine for one model

A second GB10, paired with the first (section 11), is dedicated to a single-model inference engine for Qwen3.8-27B with DFlash2, written in C++ and CUDA and outside this paper's build. Its rule is

exactness: every speed change must leave greedy output byte-identical to non-speculative decoding. It matches the production server on one stream (a step of about 103 ms) and is within 2% on two (33.4 against 33.9 tokens/s per stream). Single-step savings are kept only if bit-identical and no slower at one stream and at two. Two 16-row tiles per block in the weight-streaming GEMM were kept: the GEMMs already streamed at 233–244 GB/s at 1–16 rows, but at 17–32 rows, the batched verify of two sequences, they had fallen to 110–190 GB/s, and the change took the per-stream step period at two streams from 154.2 to 147.5 ms (32.6 to 34.1 tokens/s per stream). The commit's state replay on a second CUDA stream beside the next draft was kept: 102.6 to 101.4 ms on one stream (48.6 to 49.2 tokens/s), 147.8 to 146.3 ms per-stream period on two (34.0 to 34.3). A third `cp.async` stage in the verify attention was rejected: the kernel already read at the plain-read rate and the batched step got slower (122.3 to 124.1 ms). These comparisons are at equal acceptance because the replay now sends a sampling seed; before it did, each request drew its own acceptance, which puts noise of a few percent on every earlier figure here, 33.4 against 33.9 included, and a comparison is now reported only when both runs produce the same tokens in the same number of steps. Decode is bound by reading the weights, 17.6 GB per step, so the remaining gains are in fewer bytes read per accepted token.

11 The rest of the stack

Four further parts of Mightling follow the same method, a hook in Codex at most and the logic in a crate or service of ours. This paper only places them; each has a developer note in the repository's `docs/dev/` and a user page under `docs/`.

Unattended overnight work. *Night Shift* lets the user queue coding tasks during the day (`/night add`, optionally with the test command that decides success) and finds a branch and a report for each in the morning. The slash command writes a task record and never calls the model; the runner, in `ling-admin` under a systemd user timer, runs each task as `ling exec` in a git worktree of its own, in a memory-capped systemd scope, and itself commits; nothing is merged, pushed or rebased, and the run gives way to the user, starting only while the server is answering and idle, the host-safety checks pass and no build or index run is in progress. Live, one task has run end to end (table 3), an interrupted task has been resumed on a second night, and two tasks have run in parallel while a third waited at the memory gate (`docs/dev/night-shift.md`).

Code navigation, nodes and other surfaces. `ling-code`, a Rust binary beside `ling`, answers `def`, `refs`, `callers`, `impact` and their kin from a tree-sitter knowledge graph [3, 9, 20] that covers every language approximately and from compiler-exact SCIP [15] indexes where a front-end exists, each row tagged *exact* or *heuristic* (`docs/dev/code-index.md`); `ling-docs` indexes the user's documents without opening a socket (`docs/dev/local-file-index.md`). A machine that serves the model is a *node*, which a client finds by its identifier and never by an address, and which is managed over SSH through one forced command, never an open port (`docs/dev/node.md`). A desktop app and prebuilt macOS and Windows clients reach the same server over the local network, a browser chat is built on Onyx, and `ling web` serves a phone through a Signal, Matrix or Telegram bridge, each off until a command turns it on and named in the egress report while it is on (`docs/desktop.md`, `docs/web-chat.md`, `docs/messengers.md`).

12 Design principles

Keep the fork boring. Every behaviour that can live in a linked crate lives there, and every remaining hook is a single call or predicate. The measure of success is how long an upstream bump takes: a 43 KB series of one- and two-line hooks turns an upstream release into an afternoon's review rather than a merge project.

Presentation diffs are the expensive ones. A one-word rename inside machine-formatted data costs hundreds of kilobytes of diff; renaming at run time, or where presentation is assembled, costs nothing in the series. We expect this to be the common case for rebranded forks.

Local models change the tool contract. A harness tuned for a frontier hosted model exposes tools in ways a smaller local model may not use and a local server silently drops, so that a whole class of tools never reaches the model. It is cheaper to adapt the tool surface, by shell commands and by translating tool types at the boundary, than to wait for either side to adapt.

Measure the noise before the effect. The same configuration, run three times on the same 44 tasks, resolves between 29 and 31: 23 tasks every time, 8 never, 13 sometimes. Every candidate tested so far moves the count by three tasks or fewer, so a single run of each arm can only reject large effects, and each candidate is first judged by the behaviour it was meant to change. The instruments need the same care: the engine's replays carried sampled-acceptance noise until they were seeded, and a benchmark night whose candidate arm died at start still exited 0 until each arm was made to run one task to its end before the night begins.

Unified memory needs admission control, not just limits. A container memory cap bounds the container; it does not decide who dies when the host runs short, nor prevent a reclaim livelock in which nothing dies. Pre-flight checks, pressure-based watchdogs and caps on every auxiliary job are all necessary.

Outward-facing automation needs its own kill switch. Releases publish themselves, so a release run is started only from a release branch, signs with a key usable only there, and is checked against the repository's notes before it is dispatched.

13 Related work

SWE-agent [22] and OpenHands [21] study the agent-computer interface and evaluate on SWE-bench [11]; we do not propose a new agent, but show how to take a vendor's production client, keep it upgradable, and make it vendor-free and self-contained. LocAgent's code graphs [5], CodeNib's multi-view context serving [24] and tree-sitter knowledge graphs served over MCP [20] inform the code index (section 11). On the serving side we rely on vLLM's paged attention [12], speculative decoding [13] and block-diffusion drafting [4]; our contribution there is the host-level safety unified memory requires, not a serving algorithm.

14 Limitations

The SWE-bench figures come from 24 and 100 tasks on community images, one session per task, mapped to an overall score by models fitted on published submissions; they are not leaderboard

results, and run-to-run variation is several points. All results come from GB10 workstations with one model configuration. Agent-session throughput is a replay of recorded sessions, reaching 77% of the recorded output lengths, taken before the replay sent a sampling seed, so it carries acceptance noise of a few percent. Upstream's own suite last ran against an earlier patched tree (20,380 tests, 20,362 passed; sixteen app-server timeouts unexplained), and the slash-command suite of table 3 on a 16-patch build; both predate patches 0017 on. On SWE-bench the agent does not use the code index unprompted, and the cost of an index run on decode speed is unmeasured. Night Shift has run a handful of tasks, and a stall has not been provoked live. The advertised context still exceeds the KV pool; what the server does with a larger prompt is untested.

15 Conclusion

Mightling shows that a production coding-agent client can be made vendor-free, with its model on a single unified-memory workstation, without giving up the ability to take upstream releases. The localisation itself is about 9 KB of the 43.0 KB series; every feature beyond it (a terse-answer mode, an overnight queue, an enforced network switch, flattened MCP and Code Mode tools, connectors without a vendor sign-in, workstation pairing) costs a hook in Codex and puts its logic in crates of ours, with no change to the method. The cloud surface is audited and neutralised explicitly and re-checked by a trace after every build, and the host is protected by admission checks and a pressure watchdog. On a 100-task sample of SWE-bench Verified the agent's local result corresponds to about 69% overall; neither a study-then-fix mode, nor four prompt and harness changes, nor Code Mode's `exec` measure above the run-to-run noise. The next steps are compaction that drops old tool output before it summarises, choosing among several attempts where time allows (overnight), and speed: an engine for this one model, on the second workstation, already matches the production server, is taking milliseconds off each step with its output bit-identical, and has to pass production on seeded replays of both.

Availability

The system is developed in a public repository (`github.com/dreamference/mightling`). The patch series, launcher crates, builder and specifications described here are licensed AGPL-3.0-or-later. Codex is used under its upstream Apache-2.0 licence, which permits combination with AGPL-licensed code; its notices are retained. The Codex submodule tracks a public fork (`github.com/dgxcoder/codex`), an unmodified copy of upstream at the pinned tag.

Measurements

All numbers in this paper were measured on the author's machines or read from the repository, and each arXiv citation was checked against the arXiv record.

References

[1] Morayo Danielle Adeyemi, Ryan A. Rossi, and Franck Dernoncourt. CAVEWOMAN: How large language models behave under linguistic input and output compression. *arXiv preprint arXiv:2606.24083*, 2026.

[2] Anthropic. Model context protocol specification. `https://modelcontextprotocol.io`, 2025.

[3] Max Brunsfeld and tree-sitter contributors. Tree-sitter: an incremental parsing system for programming tools. `https://github.com/tree-sitter/tree-sitter`, 2026.

[4] Jian Chen, Yesheng Liang, and Zhijian Liu. DFlash: Block diffusion for flash speculative decoding. *arXiv preprint arXiv:2602.06036*, 2026.

[5] Zhaoling Chen, Xiangru Tang, Gangda Deng, Fang Wu, Jialong Wu, Zhiwei Jiang, Viktor Prasanna, Arman Cohan, and Xingyao Wang. LocAgent: Graph-guided LLM agents for code localization. *arXiv preprint arXiv:2503.09089*, 2025.

[6] Wenhua Cheng, Weiwei Zhang, Haihao Shen, Yiyang Cai, Xin He, Kaokao Lv, and Yi Liu. Optimize weight rounding via signed gradient descent for the quantization of LLMs. *arXiv preprint arXiv:2309.05516*, 2023.

[7] Codex contributors. Codex CLI: a lightweight coding agent that runs in your terminal. `https://github.com/openai/codex`, 2026. Release `rust-v0.158.0`, commit `064c6b8`.

[8] Deno authors. rusty_v8: Rust bindings for the V8 JavaScript engine. `https://github.com/denoland/rusty_v8`, 2026.

[9] DeusData. codebase-memory-mcp. `https://github.com/DeusData/codebase-memory-mcp`, 2026. Release v0.11.0, MIT.

[10] greynewell. SWE-bench Verified images for arm64. Docker Hub, `greynewell/swe-bench-arm64`, 2026. Community build; 400 of the 500 Verified instances.

[11] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.06770.

[12] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023. arXiv:2309.06180.

[13] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023. arXiv:2211.17192.

[14] Onyx contributors. Onyx: open-source AI chat and enterprise search. `https://github.com/onyx-dot-app/onyx`, 2026.

[15] SCIP contributors. SCIP code intelligence protocol. `https://github.com/scip-code/scip`, 2026. Apache-2.0.

[16] SearXNG contributors. SearXNG: a privacy-respecting metasearch engine. `https://github.com/searxng/searxng`, 2026.

[17] Denis Shiryaev. Does speaking to agents like cavemen really save 65% of tokens? We test. JetBrains AI Blog, `https://blog.jetbrains.com/ai/2026/07/speak-to-ai-agents-like-cavemen-tosave-tokens/`, 2026. Accessed 2026-10-01.

[18] SWE-bench team. SWE-bench experiments: published submissions and per-instance results. `https://github.com/SWE-bench/experiments`, 2026. Commit 40f164d5, 2026-09-03.

[19] The Linux Kernel documentation. PSI – pressure stall information. `https://docs.kernel.org/accounting/psi.html`.

[20] Martin Vogel, Falk Meyer-Eschenbach, Severin Kohler, Elias Grünewald, and Felix Balzer. Codebase-memory: Tree-sitter-based knowledge graphs for LLM code exploration via MCP. *arXiv preprint arXiv:2603.27277*, 2026.

[21] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. OpenHands: An open platform for AI software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024.

[22] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2405.15793.

[23] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2210.03629.

[24] Zhongming Yu, Hengjia Yu, Boqin Yuan, Shuting Zhao, Yizhao Chen, Aryan Dokania, Mihir Jagtap, Jiayu Chang, Yitong Ma, Yash Jayswal, Wentao Ni, Hejia Zhang, Zhaoling Chen, Gangda Deng, and Jishen Zhao. CodeNib: A multi-view data system for serving repository context to coding agents. *arXiv preprint arXiv:2607.25431*, 2026.

[25] z-lab and maurienne-ai. Qwen3.8-27B DFlash2 drafter, NVFP4 build. `https://huggingface.co/maurienne-ai/Qwen3.8-27B-DFlash2-NVFP4-RTNcal`, 2026. An NVFP4 build of `https://huggingface.co/z-lab/Qwen3.8-27B-DFlash2`.